

Matriz de conectividad

Grupo: 6

integrantes:

- Basualdo Alvarez, Facundo F.
- Godoy, Marianela Yasmin.
- Pare, Collante Agustina.
- Sanchez, Maria.
- Sanchez, Romina.
- Toledo, Francisco

Etapla 1 : completada con éxito

Etapla 2 - Todos deben entrar

Cada integrante instala el cliente gráfico correspondiente y se conecta al mismo servidor. Completen una matriz como el modelo de más abajo. Cada integrante debe aportar una evidencia propia de conexión.

Integrante	SO	cliente instalado	IP direccion del servidor	puerto	usuario utilizado	prueba de puerto	conexion al SGBD	Evidencia
Marianela	Windows 11	pgAdmin	100.116.70.32	5432	alumno 06	Si	Si	
Toledo Francisco	windows 10	pgAdmin	100.116.70.32	5432	alumno 04	Si	Si	
Sanchez Romina	Windows 11	pgAdmin 4 v9.17	100.116.70.32	5432	alumno 05	Sí	Sí	
Sanchez Maria Eugenia	Windows 10	PgAdmin 4v9.17	100.116.70.32	5432	alumno 02	Si	Si	
Pare Collante,	Windows 11	PgAdmin 4v9.17	100.116.70.32	5432	alumno 03	Si	Si	

Agustina								
Basualdo Alvarez Facundo	wd 11 home	pgAdmin 9.17	100.116.70.32	5432	alumno 01	si		

Evidencias:

Agustina Paré

Prueba del puerto

```

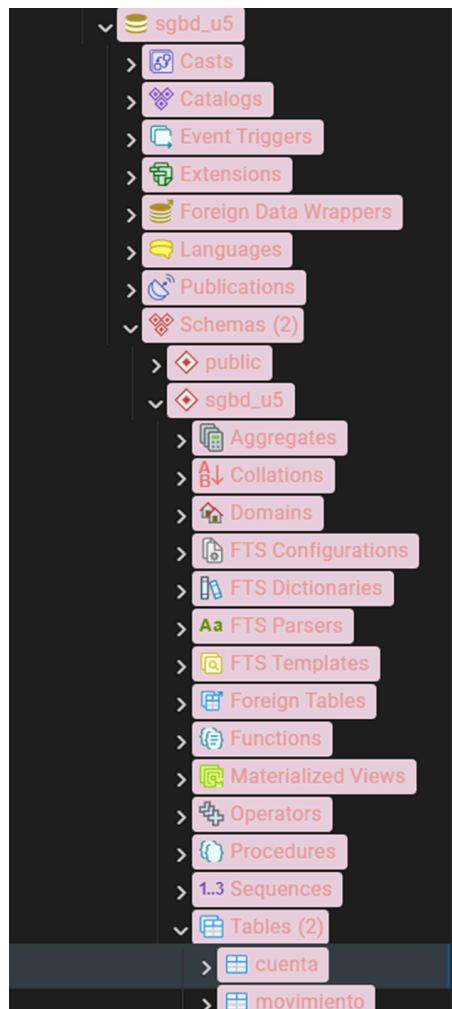
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\tinip> Test-NetConnection 100.116.70.32 -Port 5432

ComputerName      : 100.116.70.32
RemoteAddress     : 100.116.70.32
RemotePort        : 5432
InterfaceAlias    : Tailscale
SourceAddress     : 100.96.137.113
TcpTestSucceeded  : True

```

pgAdmin conectado



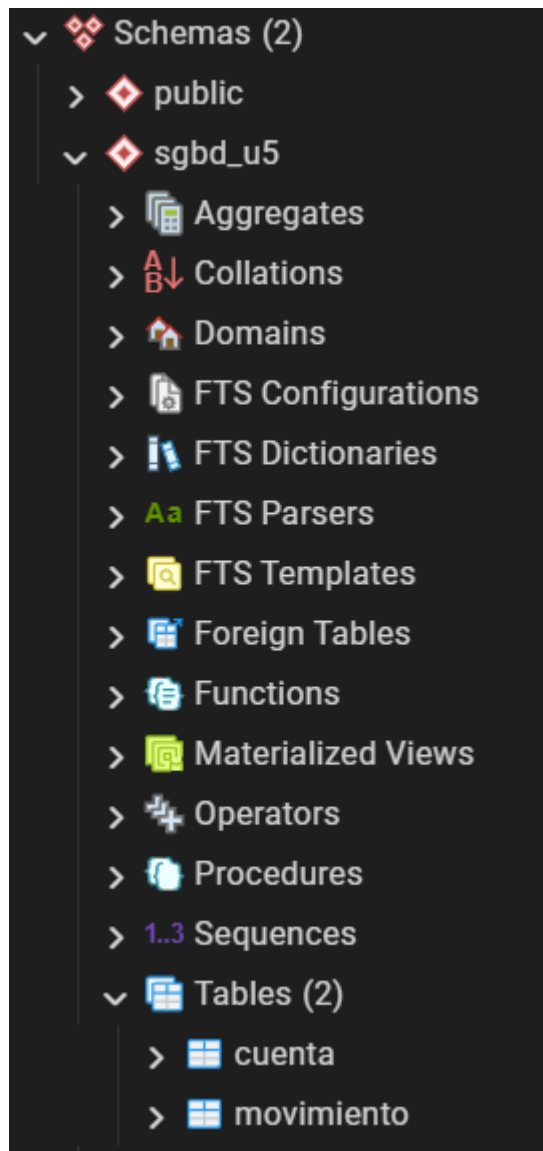
Facundo Basualdo

prueba de puerto:

```
PS C:\Users\facun> Test-NetConnection 100.116.70.32 -Port 5432

ComputerName      : 100.116.70.32
RemoteAddress     : 100.116.70.32
RemotePort        : 5432
InterfaceAlias    : Tailscale
SourceAddress     : 100.96.254.31
TcpTestSucceeded  : True
```

pgadmin conectado:



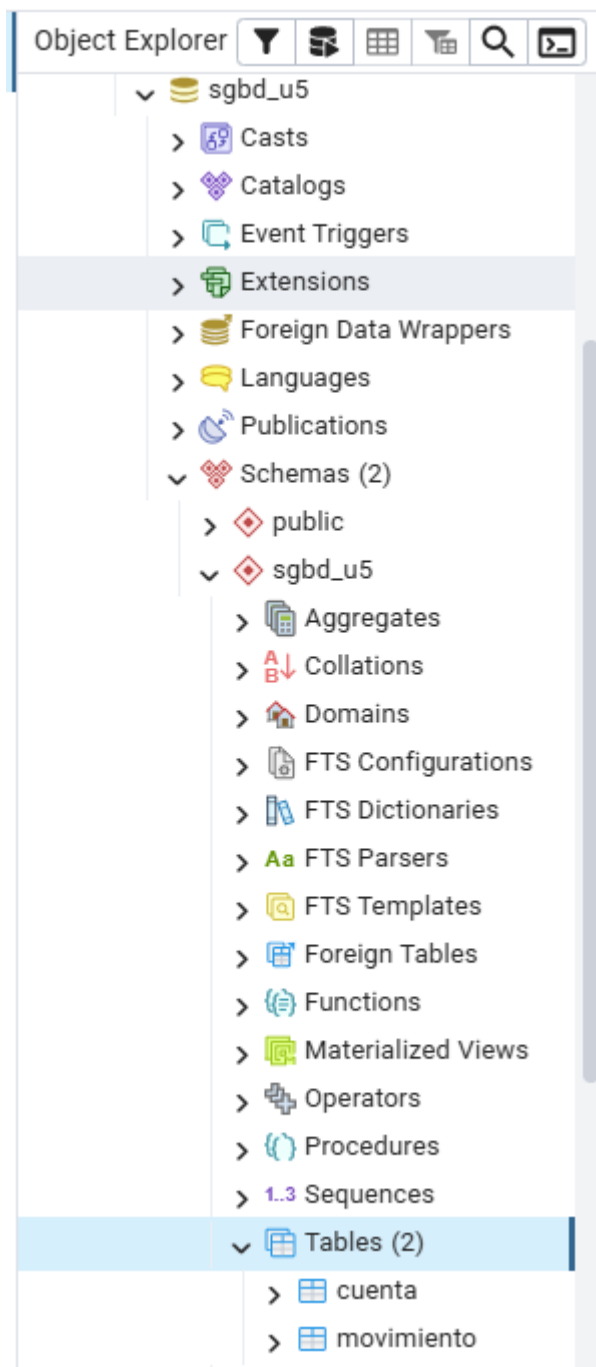
Maria Eugenia Sanchez:

CAPTURA A:

```
PS C:\Users\Usuario> Test-NetConnection 100.116.70.32 -Port 5432

ComputerName      : 100.116.70.32
RemoteAddress     : 100.116.70.32
RemotePort        : 5432
InterfaceAlias    : Tailscale
SourceAddress     : 100.71.110.74
TcpTestSucceeded  : True
```

CAPTURA B:



CAPTURA C:

	current_user name	session_user name	current_database name	version text
1	alumno02	alumno02	sgbd_u5	PostgreSQL 18.6 (Ubuntu 18.6-0ubuntu0.26.04.1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, 64-bit

Marianela Yasmin Godoy

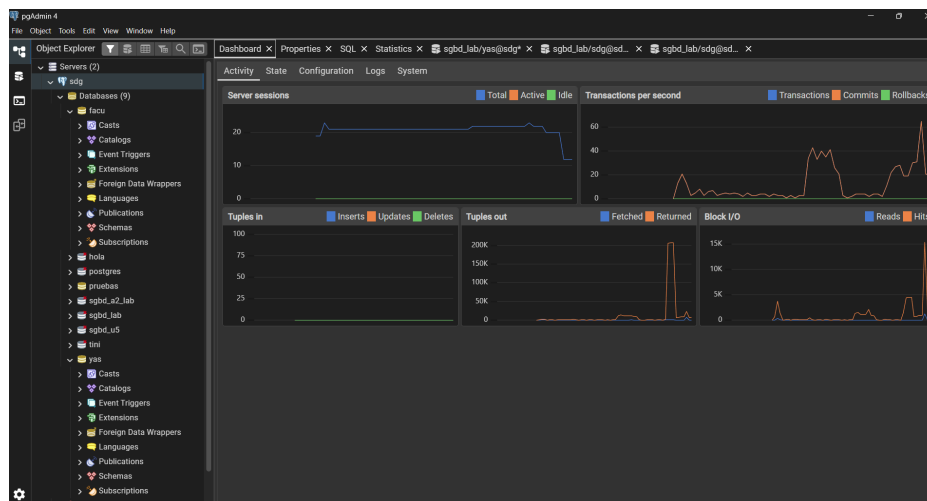
Evidencia 1: Demuestra el acceso operativo a la máquina virtual, la verificación de red y el entorno del sistema.

```
PS C:\Users\maria> Test-NetConnection 100.116.70.32 -Port 5432

ComputerName      : 100.116.70.32
RemoteAddress     : 100.116.70.32
RemotePort        : 5432
InterfaceAlias    : Tailscale
SourceAddress     : 100.104.243.100
TcpTestSucceeded  : True

PS C:\Users\maria>
```

Evidencia 2: pgAdmin demuestra la conectividad exitosa del cliente gráfico



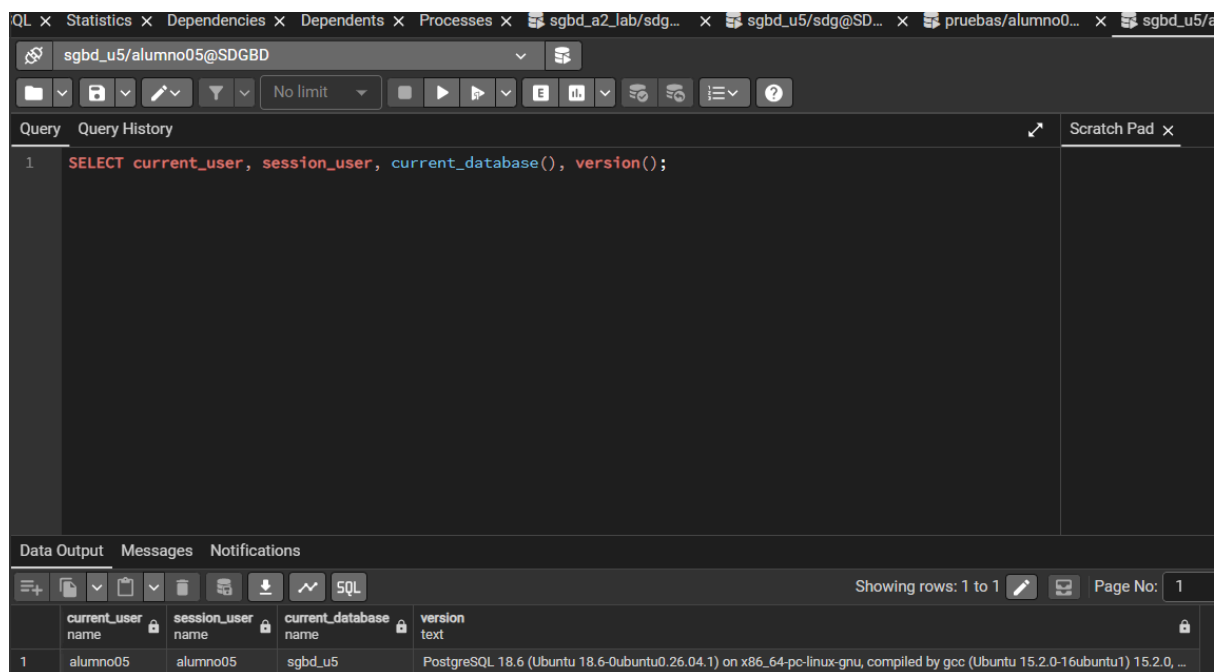
Romina Sanchez:

Evidencia 1:

```
PS C:\WINDOWS\system32> Test-NetConnection 100.116.70.32 -Port 5432

ComputerName      : 100.116.70.32
RemoteAddress     : 100.116.70.32
RemotePort        : 5432
InterfaceAlias     : Tailscale
SourceAddress     : 100.66.1.80
TcpTestSucceeded  : True
```

Evidencia 2:



The screenshot shows a PostgreSQL client interface with a query editor and a results pane. The query editor contains the following SQL query:

```
1 SELECT current_user, session_user, current_database(), version();
```

The results pane displays the output of the query in a table format:

	current_user name	session_user name	current_database name	version text
1	alumno05	alumno05	sgbd_u5	PostgreSQL 18.6 (Ubuntu 18.6-0ubuntu0.26.04.1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, ...

Francisco Toledo

Captura A

```
PS C:\Users\Educacion> Test-NetConnection 100.116.70.32 -port 5432

ComputerName      : 100.116.70.32
RemoteAddress     : 100.116.70.32
RemotePort        : 5432
InterfaceAlias     : Tailscale
SourceAddress     : 100.83.194.88
TcpTestSucceeded  : True
```

Captura B

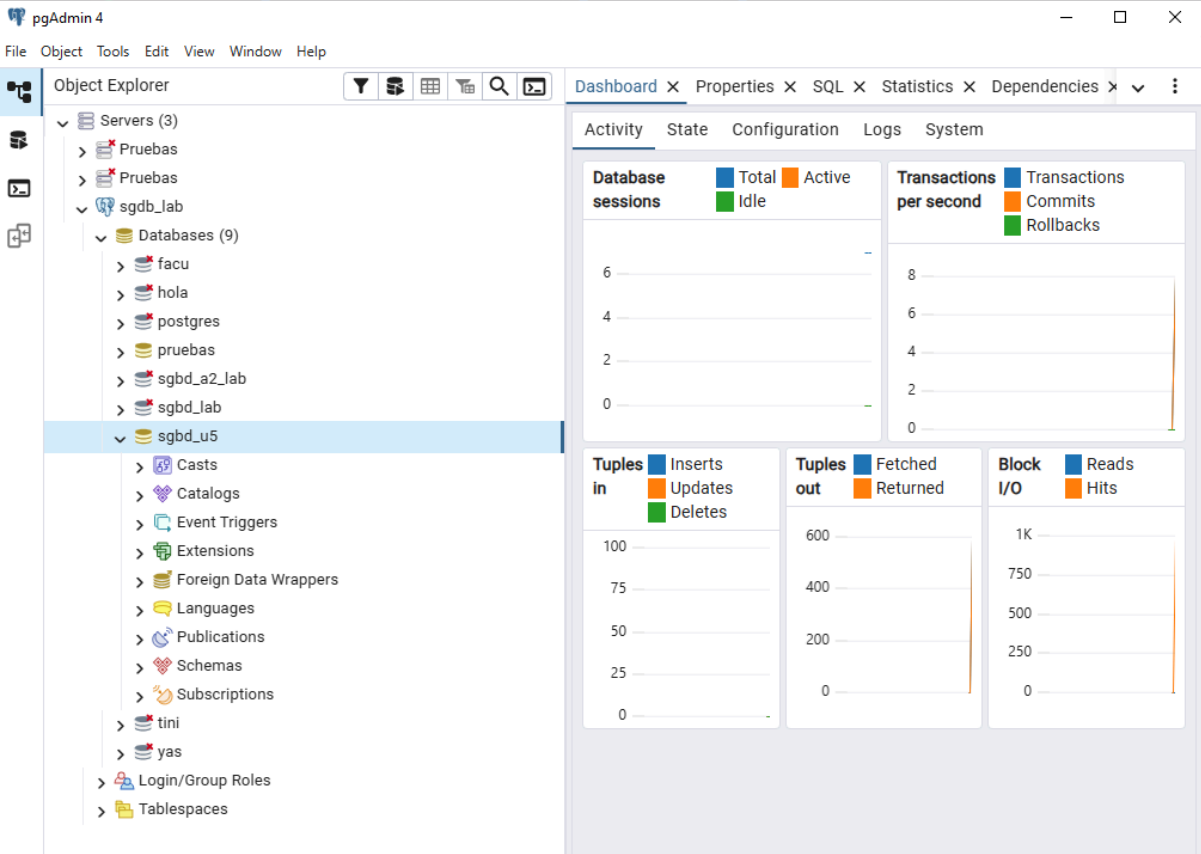
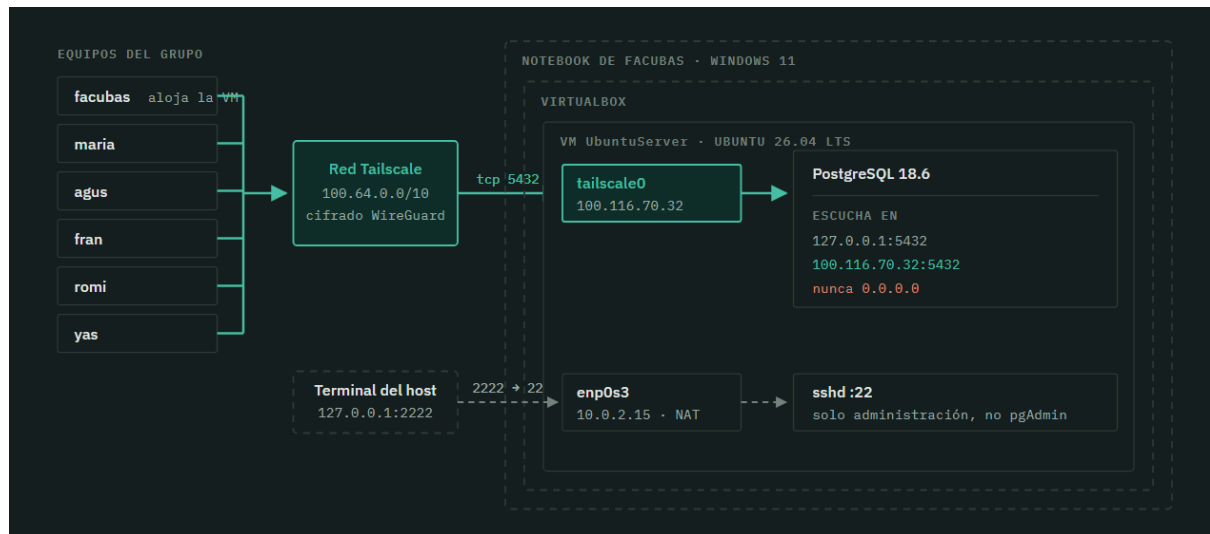


Diagrama de arquitectura

Diagrama de arquitectura real: equipos del grupo -> red -> VM -> SGBD.

Gráfico:



El tráfico del grupo entra a la VM directamente por Tailscale, sin usar el port forward de VirtualBox ni una regla en el firewall de Windows.

filtros:



Abrir el servidor a la red no expuso las demás bases: pasan dos controles distintos y solo sgbd_u5 atraviesa los dos.

parámetros reales:

Elemento	Valor
Virtualización	VirtualBox · red NAT
Sistema de la VM	Ubuntu 26.04 LTS
Motor	PostgreSQL 18.6
Nombre en la tailnet	vm-sdg
IP Tailscale de la VM	100.116.70.32
IP NAT interna	10.0.2.15
Puerto del motor	5432
Direcciones de escucha	127.0.0.1 y 100.116.70.32
Regla de acceso	host sgbd_u5 all 100.64.0.0/10 scram-sha-256
Base del laboratorio	sgbd_u5
Cuentas	alumno01 ... alumno06 · rol pg_monitor

POR QUÉ NO USAMOS ADAPTADOR PUENTE:

La guía de la cátedra recomienda bridged. La consigna acepta otras soluciones de red siempre que sean explicables, reproducibles y seguras, y el apartado 9 del instructivo contempla una VPN privada. Estas son las razones de la elección:

1. No depende de la VLAN del aula. La red wfre-2 entrega un /21, y si un integrante cae en otra subred queda afuera. La tailnet es la misma esté donde esté cada uno.
2. Es más restrictivo que lo que pedía la guía. En vez de listen_addresses = '*', el motor escucha en dos direcciones concretas, y nunca en la red del aula.
3. Funciona igual desde casa. No hay que abrir puertos en el router ni publicar 5432 en internet, que es justamente lo que el instructivo desaconseja.
4. Utilizamos esta metodología desde la primera clase.

Checklist DBA

Checklist DBA completo y distribuido entre integrantes.

Integrante	Cuenta	Tarea propia (además de lo que hacen todos)
facubas	alumno01	Diagrama de arquitectura + armar el PDF final
maria	alumno02	Etapa 3 — Objetos
agus	alumno03	Etapa 3 — Seguridad
fran	alumno04	Etapa 3 — Metadatos
romi	alumno05	Etapa 3 — Ingeniería inversa (ERD)
yas	alumno06	Etapa 5 — Plan de ejecución

Datos de conexión, iguales para todos: host 100.116.70.32, puerto 5432, base sgbd_u5, contraseña SgbdU5_2026!.

Etapa 2-Objetos

OBJETOS:

```
SELECT CASE c.relkind
        WHEN 'r' THEN 'tabla' WHEN 'v' THEN 'vista'
        WHEN 'S' THEN 'secuencia' WHEN 'i' THEN 'indice'
        END          AS tipo,
        c.relname     AS nombre
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'sgbd_u5'
ORDER BY 1, 2;
```

	tipo text	nombre name
1	indice	cuenta_pkey
2	indice	ix_mov_cuenta_fecha
3	indice	movimiento_pkey
4	secuenc...	movimiento_movimiento_id_s...
5	tabla	cuenta
6	tabla	movimiento

Este query revisa el catálogo de postgres para listar todos los objetos principales (tablas, vistas, secuencias e índices), renombrando los tipos de objeto a un formato legible.

pg_class (c) : Catálogo del sistema que almacena información sobre la estructura de tablas y objetos similares (índices, secuencias, vistas).

pg_namespace (n): Catálogo que almacena los esquemas de la base de datos.

```
SELECT (SELECT count(*) FROM pg_proc p
        JOIN pg_namespace n ON n.oid = p.pronamespace
        WHERE n.nspname = 'sgbd_u5') AS
funciones,
        (SELECT count(*) FROM pg_trigger t
        JOIN pg_class c ON c.oid = t.tgrelid
        JOIN pg_namespace n ON n.oid = c.relnamespace
        WHERE n.nspname = 'sgbd_u5' AND NOT t.tgisinternal) AS
triggers;
```

	funciones bigint	triggers bigint
1	0	0

Este query calcula y muestra, la cantidad total de funciones y la cantidad total de triggers de usuario que existen dentro del esquema 'sgbd_u5', en nuestro caso no hay.

```
SELECT conrelid::regclass AS tabla, conname AS nombre,
       CASE contype WHEN 'p' THEN 'PRIMARY KEY' WHEN 'f' THEN 'FOREIGN
KEY'
              WHEN 'c' THEN 'CHECK' WHEN 'n' THEN 'NOT NULL' END
AS tipo
FROM pg_constraint
WHERE connamespace = 'sgbd_u5'::regnamespace
ORDER BY 1, 3;
```

	tabla regclass	nombre name	tipo text
1	cuenta	cuenta_saldo_check	CHECK
2	cuenta	cuenta_cuenta_id_not_null	NOT NULL
3	cuenta	cuenta_titular_not_null	NOT NULL
4	cuenta	cuenta_saldo_not_null	NOT NULL
5	cuenta	cuenta_estado_not_null	NOT NULL
6	cuenta	cuenta_pkey	PRIMARY K...
7	movimien...	movimiento_cuenta_id_fkey	FOREIGN KEY
8	movimien...	movimiento_cuenta_id_not_null	NOT NULL
9	movimien...	movimiento_fecha_not_null	NOT NULL
10	movimien...	movimiento_tipo_not_null	NOT NULL
11	movimien...	movimiento_importe_not_null	NOT NULL
12	movimien...	movimiento_movimiento_id_not_...	NOT NULL
13	movimien...	movimiento_pkey	PRIMARY K...

Este query obtiene el listado de todas las restricciones de integridad (*constraints*) definidas dentro del esquema 'sgbd_u5', mostrando a qué tabla pertenecen, cómo se llaman y de qué tipo son.

La PK identifica cada fila sin repetidos, la FK obliga a que todo movimiento pertenezca a una cuenta que exista, el CHECK impide que un saldo quede negativo.

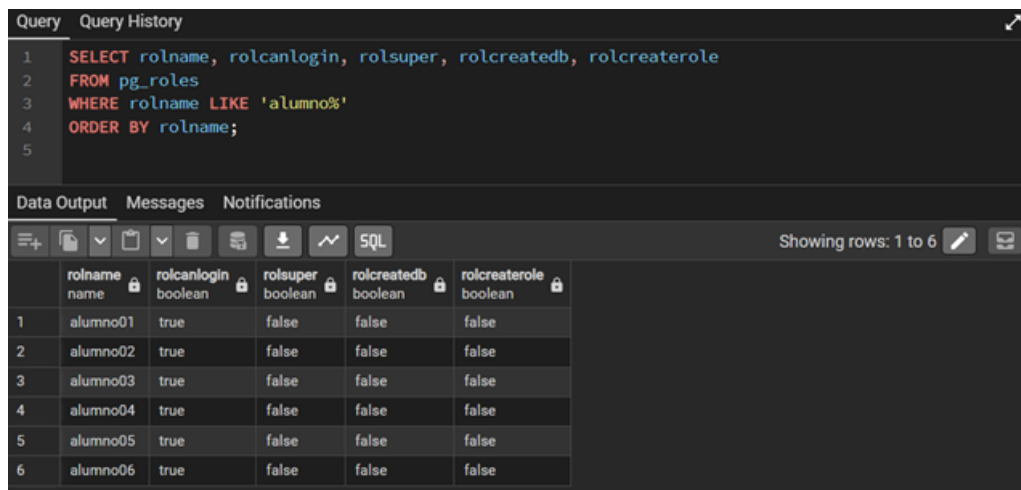
Etapa 3 — Seguridad

Seguridad

Roles de las seis cuentas del grupo

Dónde está: Login/Group Roles → cada rol alumnoXX → solapa Properties

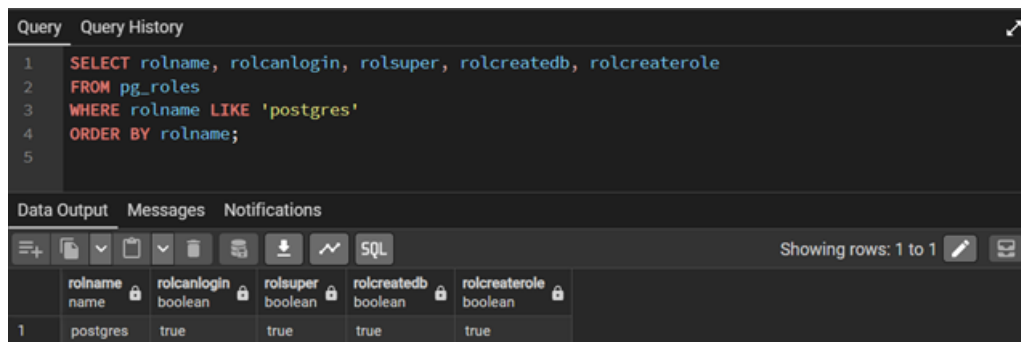
```
SELECT rolname, rolcanlogin, rolsuper, rolcreatedb, rolcreaterole
FROM pg_roles
WHERE rolname LIKE 'alumno%'
ORDER BY rolname;
```



The screenshot shows a SQL query editor with a query history tab. The query is: `SELECT rolname, rolcanlogin, rolsuper, rolcreatedb, rolcreaterole FROM pg_roles WHERE rolname LIKE 'alumno%' ORDER BY rolname;`. The results are displayed in a table with 6 rows and 5 columns.

	rolname name	rolcanlogin boolean	rolsuper boolean	rolcreatedb boolean	rolcreaterole boolean
1	alumno01	true	false	false	false
2	alumno02	true	false	false	false
3	alumno03	true	false	false	false
4	alumno04	true	false	false	false
5	alumno05	true	false	false	false
6	alumno06	true	false	false	false

Las seis cuentas pueden iniciar sesión pero ninguna es superusuario ni puede crear bases o roles nuevos. Para contrastar, así se ve postgres, el superusuario real del servidor:



The screenshot shows a SQL query editor with a query history tab. The query is: `SELECT rolname, rolcanlogin, rolsuper, rolcreatedb, rolcreaterole FROM pg_roles WHERE rolname LIKE 'postgres' ORDER BY rolname;`. The results are displayed in a table with 1 row and 5 columns.

	rolname name	rolcanlogin boolean	rolsuper boolean	rolcreatedb boolean	rolcreaterole boolean
1	postgres	true	true	true	true

Para qué le sirve a un DBA: Es la primera pregunta de seguridad ante cualquier cuenta: puede entrar, sí, pero ¿hasta dónde llega? El contraste alumnoXX / postgres deja ver en dos filas la diferencia entre una cuenta de trabajo y el superusuario del motor.

Pertenencia a pg_monitor

Dónde está: Login/Group Roles → alumnoXX → Properties → solapa Membership

```

SELECT r.rolname AS miembro, g.rolname AS pertenece_a
FROM pg_auth_members m
JOIN pg_roles r ON r.oid = m.member
JOIN pg_roles g ON g.oid = m.roleid
WHERE r.rolname LIKE 'alumno%'
ORDER BY 1;

```

Query Query History

```

1 SELECT r.rolname AS miembro, g.rolname AS pertenece_a
2 FROM pg_auth_members m
3 JOIN pg_roles r ON r.oid = m.member
4 JOIN pg_roles g ON g.oid = m.roleid
5 WHERE r.rolname LIKE 'alumno%'
6 ORDER BY 1;
7

```

Data Output Messages Notifications

	miembro name	pertenece_a name
1	alumno01	pg_monitor
2	alumno02	pg_monitor
3	alumno03	pg_monitor
4	alumno04	pg_monitor
5	alumno05	pg_monitor
6	alumno06	pg_monitor

Qué muestra: Que las seis cuentas están agregadas al rol predefinido pg_monitor.

Para qué le sirve a un DBA: pg_monitor es un rol de PostgreSQL pensado exactamente para esto: observar el servidor se puede ver sesiones activas y estadísticas de uso, sin poder modificar nada. Es la idea de mínimo privilegio aplicada a la guardia: quien está de turno mira, no toca. Es lo que en la Parte 2 me permitió ver, sin ser superusuario, la consulta que estaba corriendo otra sesión.

Permisos sobre las tablas

Dónde está: Click derecho sobre cuenta o movimiento → Properties → Security

```

SELECT c.relname AS tabla, unnest(c.relacl)::text AS permiso
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'sgbd_u5' AND c.relkind = 'r'

```

```
ORDER BY 1, 2;
```

Query

Query History

1

SELECT c.relname AS tabla, unnest(c.relacl)::text AS permiso

2

FROM pg_class c

3

JOIN pg_namespace n ON n.oid = c.relnamespace

4

WHERE n.nspname = 'sgbd_u5' AND c.relkind = 'r'

5

ORDER BY 1, 2;

Data Output

Messages

Notifications

</

Qué muestra: Cada alumnoXX=arwd/postgres se lee: el rol tiene a=INSERT, r=SELECT, w=UPDATE, d=DELETE, otorgados por postgres. El rol sdg (de una actividad anterior de la materia) aparece con más letras, arwdDxtm, igual que el propio postgres como dueño.

Para qué le sirve a un DBA: Confirma que el grupo tiene exactamente los permisos de trabajo (leer y escribir datos) y ninguno de administración (no puede borrar la tabla ni cambiar su estructura). Es la contraparte a nivel de datos del mismo principio de mínimo privilegio que se ve en pg_monitor a nivel de servidor.

Etapa 3 - Metadatos

Etapa 3 - Reconocer el territorio DBA

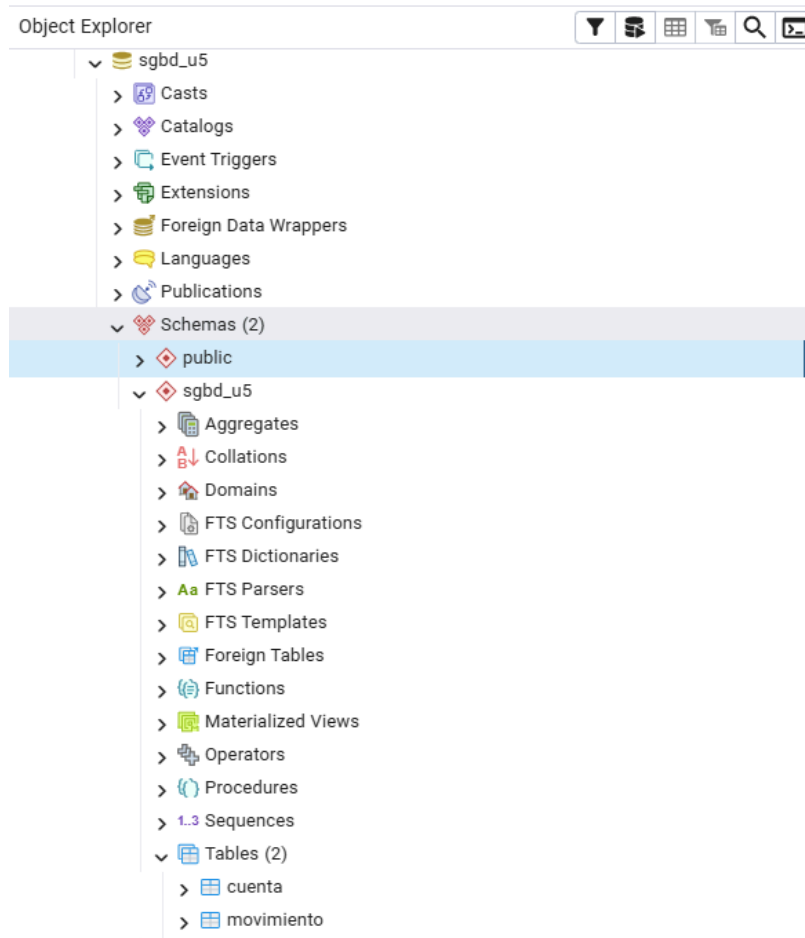
Distribuyan el checklist entre los integrantes. Para cada punto respondan: **donde esta, que muestra, para que le sirve a un DBA y que evidencia obtuvieron.**

Area	Investigacion
Objetos	Bases/esquemas, tablas, vistas, indices, constraints, funciones/procedimientos y triggers disponibles. Describir brevemente cada uno.
Seguridad	Usuarios/roles y privilegios visibles desde la herramienta.
Metadatos	Columnas, tipos, PK, FK, indices y propiedades de una tabla elegida. Explicar que información de utilidad brinda y para que casos la usaría.
Ingeniería inversa	Generar un ERD/modelo a partir de una base o esquema existente. Pegar Diagrama obtenido

Metadatos

¿Dónde está?:

-Primero localizamos la tabla de movimientos a través del buscador de pgAdmin



-Luego hacemos click derecho en la tabla movimiento, seleccionamos propiedades

movimiento

General Columns Advanced Constraints Partitions Parameters Security SQL

Inherited from table(s)

Columns							
	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
	movimiento_id	bigint			☒	☒	
	cuenta_id	integer			☒	☐	
	fecha	timestamp with tzn			☒	☐	CURRENT
	tipo	character varying	20		☒	☐	
	importe	numeric	12	2	☒	☐	

-Desde esta ventana podemos ver las columnas, constraints, PK, FK y propiedades de la tabla de movimiento. Sin embargo no se pueden visualizar los índices.

-Por lo tanto para poder ver los índices existentes ejecutamos la siguiente sentencia:

Query Query History

```

1 SELECT indexname AS indice, indexdef AS definicion
2 FROM pg_indexes
3 WHERE schemaname = 'sgbd_u5' AND tablename = 'movimiento';

```

-Y como resultado podemos ver:

	indice name	definicion text
1	movimiento_pkey	CREATE UNIQUE INDEX movimiento_pkey ON sgbd_u5.movimiento USING btree (movimie...
2	ix_mov_cuenta_fec...	CREATE INDEX ix_mov_cuenta_fecha ON sgbd_u5.movimiento USING btree (cuenta_id, fe...

-Existe un índice llamado ix_mov_cuenta_fecha (cuenta_id, fecha)

¿Qué información de utilidad brinda cada dato?

- Columnas: nos indican los tipos de datos que maneja cada columna de la tabla y cuanto ocupa. Es importante saber los tipos de datos de cada columna para saber cómo trabajar correctamente con los datos. Por ejemplo la columna importe se define específicamente como numeric (12, 2), en vez de utilizar float, con el supuesto de que se debe evitar usar float en importes numéricos ya que float puede redondear los decimales, de manera que se guarden datos erróneos.
- PK: La clave primaria movimiento_id, existe para garantizar que no existan dos tuplas de movimientos iguales.

- FK: La clave foranea cuenta impide que exista un movimiento de una cuenta que no existe.
- El índice ix_mov_cuenta_fecha sobre (cuenta_id, fecha), sirven como atajo para buscar los movimientos de x cuenta en periodo.

¿Cómo le sirven estos datos a un DBA?

-En este caso, para la tabla de movimiento, la herramienta que más le sirve a un DBA es conocer de antemano la existencia de un índice. Un índice ayuda en la optimización de consultas de mayor volumen, permitiendo evaluar alternativas ante consultas pesadas en procesamiento.

Evidencias

Para evidenciar la existencia de los tipos de datos, PK, FK e índices se ejecutan las siguientes sentencias:

Columnas y tipos de datos:

```
SELECT ordinal_position AS orden, column_name AS columna,
       data_type AS tipo, is_nullable AS acepta_nulos,
       character_maximum_length AS largo_max, numeric_precision AS precision
FROM information_schema.columns
WHERE table_schema = 'sgbd_u5' AND table_name = 'movimiento'
ORDER BY ordinal_position;
```

Resultado:

	orden integer	columna name	tipo character varying	acepta_nulos character varying (3)	largo_max integer	precision integer
1	1	movimiento...	bigint	NO	[null]	64
2	2	cuenta_id	integer	NO	[null]	32
3	3	fecha	timestamp with time zo...	NO	[null]	[null]
4	4	tipo	character varying	NO	20	[null]
5	5	importe	numeric	NO	[null]	12

PK y FK:

```
SELECT conname AS nombre,
       CASE contype WHEN 'p' THEN 'PRIMARY KEY' WHEN 'f' THEN 'FOREIGN KEY' END AS tipo,
       pg_get_constraintdef(oid) AS definicion
FROM pg_constraint
WHERE conrelid = 'sgbd_u5.movimiento'::regclass AND contype IN ('p','f');
```

Resultado:

	nombre name	tipo text	definicion text
1	movimiento_pkey	PRIMARY K...	PRIMARY KEY (movimiento_id)
2	movimiento_cuenta_id_fk...	FOREIGN K...	FOREIGN KEY (cuenta_id) REFERENCES cuenta(cuenta...

Índices:

```
SELECT indexname AS indice, indexdef AS definicion
FROM pg_indexes
WHERE schemaname = 'sgbd_u5' AND tablename = 'movimiento';
```

Resultado:

	índice name	definición text
1	movimiento_pkey	CREATE UNIQUE INDEX movimiento_pkey ON sgbd_u5.movimiento USING btree (movimie...
2	ix_mov_cuenta_fec...	CREATE INDEX ix_mov_cuenta_fecha ON sgbd_u5.movimiento USING btree (cuenta_id, fe...

Etapa 3 — Ingeniería inversa (ERD)



- **Relación:** Se trata de una relación uno a muchos, lo que significa que una cuenta tiene muchos movimientos, pero cada movimiento pertenece a exactamente una sola cuenta.

- **Identificadores:** La llave de la izquierda marca la Clave Primaria (PK) de cada tabla.

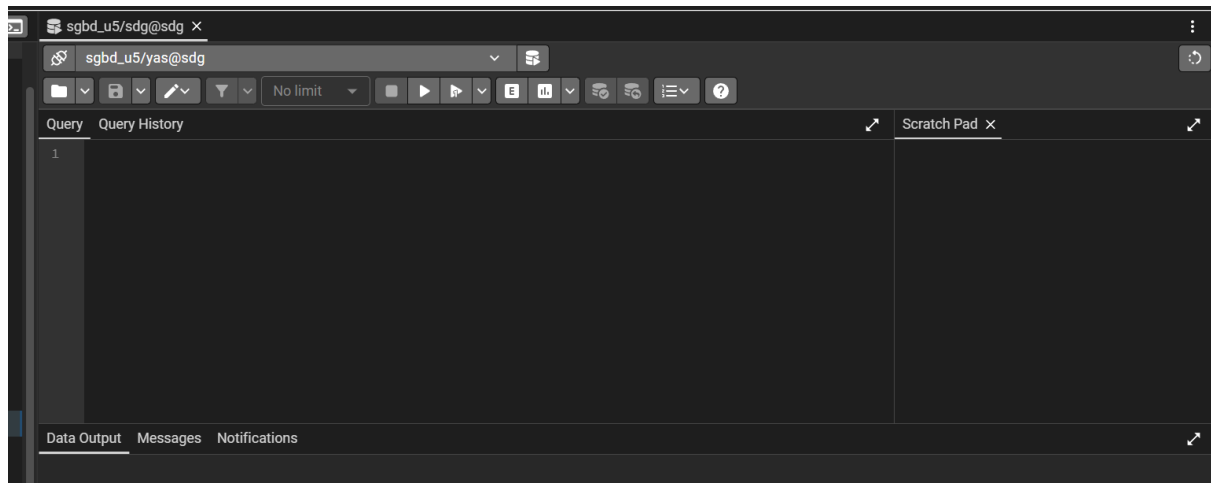
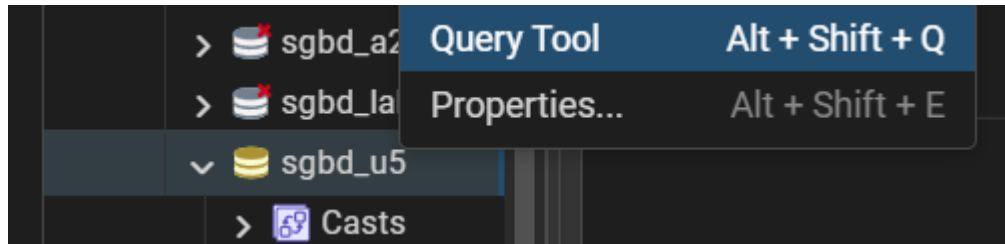
- **Utilidad para un DBA (Ingeniería Inversa):**

Cuando un DBA recibe una base de datos que no conoce y no cuenta con documentación, generar el ERD le permite entender toda la estructura en minutos en vez de tener que leer la definición tabla por tabla.

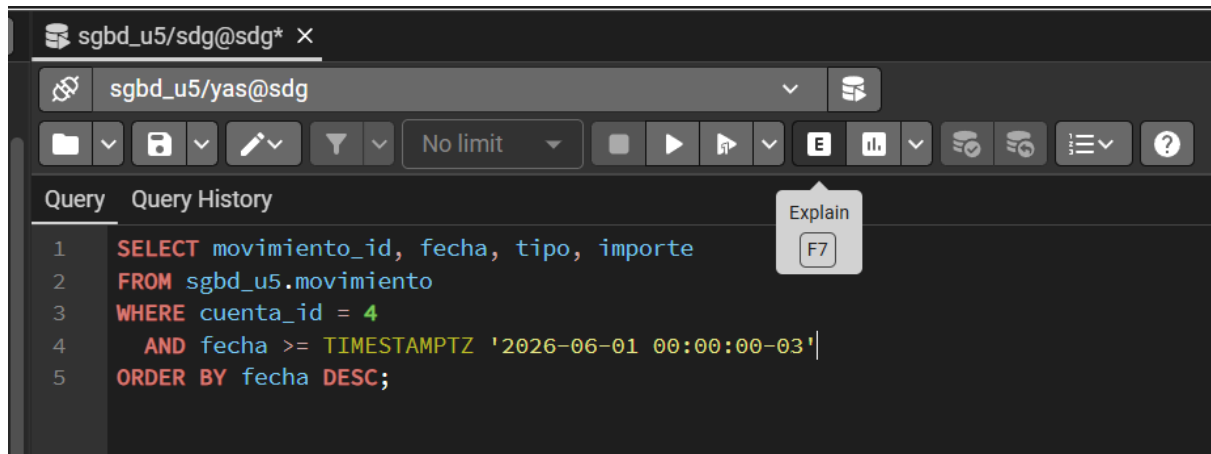
Etapa 5 - Plan de Ejecucion

Plan de ejecución - Etapa 5

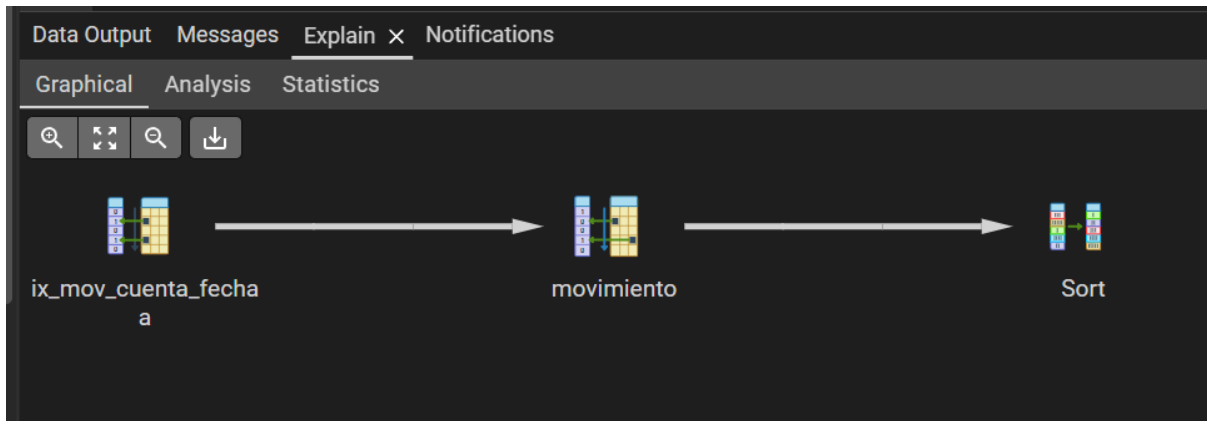
Paso 1: Buscar la base de datos de la materia de esta actividad.



Paso 2: Consulta de movimientos recientes y se copia la consulta y luego Explain Analyze



Resultado:



Ejecutando el **EXPLAIN (ANALYZE, BUFFERS)** provisto en el script, se puede ver el plan detallado en texto:

sgbd_u5/sdg@sdg* X

sgbd_u5/yas@sdg

Query Query History

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT movimiento_id, fecha, tipo, importe
3 FROM sgbd_u5.movimiento
4 WHERE cuenta_id = 4
5 AND fecha >= TIMESTAMPTZ '2026-06-01 00:00:00-03'
6 ORDER BY fecha DESC;
```

Data Output Messages Explain X Notifications

Showing rows: 1 to 14

	QUERY PLAN
1	Sort (cost=1881.97..1885.37 rows=1361 width=33) (actual time=1.373..1.439 rows=1370.00 loops=1)
2	Sort Key: fecha DESC
3	Sort Method: quicksort Memory: 119kB
4	Buffers: shared hit=477
5	-> Bitmap Heap Scan on movimiento (cost=38.37..1811.13 rows=1361 width=33) (actual time=0.292..1.175 rows=1370.00 loops=1)
6	Recheck Cond: ((cuenta_id = 4) AND (fecha >= '2026-06-01 03:00:00+00':timestamp with time zone))
7	Heap Blocks: exact=469
8	Buffers: shared hit=477
9	-> Bitmap Index Scan on ix_mov_cuenta_fecha (cost=0.00..38.03 rows=1361 width=0) (actual time=0.099..0.099 rows=1370.00 loops=1)
10	Index Cond: ((cuenta_id = 4) AND (fecha >= '2026-06-01 03:00:00+00':timestamp with time zone))
11	Index Searches: 1
12	Buffers: shared hit=8
13	Planning Time: 0.084 ms
14	Execution Time: 1.529 ms

Paso 3: Consulta de totales (la pesada) se hace ejecutando y obteniendo el plan de la Consulta B.

Pegando la segunda consulta y ejecutando el EXplain:

The screenshot shows the DBeaver SQL editor interface. The top toolbar includes icons for file operations, query execution, and settings. The main query area contains the following SQL code:

```
1 SELECT cuenta_id, tipo, count(*), sum(importe)
2 FROM sgbd_u5.movimiento
3 GROUP BY cuenta_id, tipo
4 ORDER BY cuenta_id, tipo;
```

Below the query editor, the 'Query History' tab is active, showing the same query. To the right, a 'Scratch Pad' tab is visible. At the bottom, the 'Data Output' tab is selected, displaying the 'Explain' view of the query execution plan. The plan shows the following steps:

- movimiento**: The initial data source, represented by a table icon.
- Aggregate**: The first aggregation step, represented by a table icon with a plus sign.
- Sort**: The sorting step, represented by a table icon with a downward arrow.
- Gather Merge**: The gathering and merging step, represented by a table icon with multiple arrows.
- Aggregate**: The final aggregation step, represented by a table icon with a plus sign.

Estado del servidor

Estado del servidor y tres indicadores interpretados.

Siete puntos, todos sacados desde el Query Tool de mi propia sesión, salvo b) y c) que dependen de la ventana de guardia con el grupo conectado a la vez.

a) Servidor

```
SELECT current_user AS usuario_conectado,  
current_database() AS base_analizada,  
version() AS motor_y_version;
```



The screenshot shows a PostgreSQL Query Tool window. The 'Query' tab is active, displaying the following SQL query:

```
1 SELECT current_user AS usuario_conectado,  
2 current_database() AS base_analizada,  
3 version() AS motor_y_version;  
4
```

The 'Data Output' tab is also visible, showing the results of the query in a table with three columns: 'usuario_conectado', 'base_analizada', and 'motor_y_version'. The results are as follows:

usuario_conectado	base_analizada	motor_y_version
alumno03	sgbd_u5	PostgreSQL 18.6 (Ubuntu 18.6-0ubuntu0.26.04.1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, 2015.12.17

usuario_conectado: alumno03

Quedé conectada como **alumno03** contra **sgbd_u5**, corriendo PostgreSQL 18.6 sobre Ubuntu.

b) Sesiones

PENDIENTE — VENTANA DE GUARDIA

Se completa cuando los seis estemos conectados a la vez y cada uno corra la consulta de abajo. Tienen que aparecer seis filas, una por integrante. Si me conecto sola solo veo la mía, así que esta evidencia no se puede adelantar.

```
SELECT pid, username, client_addr, state, backend_start  
FROM pg_stat_activity  
WHERE datname = 'sgbd_u5'  
ORDER BY username;
```

c) Actividad

```
SELECT pid, username, client_addr, state, wait_event_type,  
left(query, 60) AS ejecutando,  
now() - query_start AS duracion  
FROM pg_stat_activity  
WHERE datname = 'sgbd_u5' AND state = 'active';
```

Query

Query History

1

2

3

4

5

6

SELECT pid, username, client_addr, state, wait_event_type,

left(query, 60) AS ejecutando,

now() - query_start AS duracion

FROM pg_stat_activity

WHERE datname = 'sgbd_u5' AND state = 'active';

Data Output

Messages

Notifications

<

d) Tabla

Elegí movimiento por sobre cuenta: tiene 200.000 filas contra apenas 4.

```
SELECT count(*) AS filas FROM sgbd_u5.movimiento;

SELECT indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'sgbd_u5' AND tablename = 'movimiento';
```

Query

Query History

1

SELECT count(*) AS filas FROM sgbd_u5.movimiento;

2

SELECT indexname, indexdef

3

FROM pg_indexes

4

WHERE schemaname = 'sgbd_u5' AND tablename = 'movimiento';

5

6

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑

📦

📥

📈

SQL

Showing results

	indexname name	indexdef text
1	movimiento_pkey	CREATE UNIQUE INDEX movimiento_pkey ON sgbd_u5.movimiento USING btree (movimiento_id)
2	ix_mov_cuenta_fec...	CREATE INDEX ix_mov_cuenta_fecha ON sgbd_u5.movimiento USING btree (cuenta_id, fecha)

movimiento_pkey

movimiento tiene dos índices: movimiento_pkey, que se crea solo al definir la clave primaria, e ix_mov_cuenta_fecha, creado a mano sobre dos columnas juntas. Ese segundo índice está pensado para la consulta típica “movimientos de una cuenta en un período”, que es justo la que corre yas en la Etapa 5.

e) Rendimiento

```
SELECT round(100.0*blks_hit/nullif(blks_hit+blks_read,0), 2) AS cache_hit_pct,
       xact_commit,
       xact_rollback,
       deadlocks,
       temp_files
FROM pg_stat_database
WHERE datname = 'sgbd_u5';
```

Query

Query History

1

SELECT round(100.0*blks_hit/nullif(blks_hit+blks_read,0), 2) AS cache_hit_pct,

2

xact_commit,

3

xact_rollback,

4

deadlocks,

5

temp_files

6

FROM pg_stat_database

7

WHERE datname = 'sgbd_u5';

8

Data Output

Messages

Notifications

≡

+

SQL

Showing rows: 1 to 1

	cache_hit_pct numeric	xact_commit bigint	xact_rollback bigint	deadlocks bigint	temp_files bigint	
1	99.89	1407	6	0	1	

Rendimiento

En ninguna de las dos tablas corrió nunca un VACUUM, ni manual ni automático (last_vacuum y last_autovacuum vacíos). ANALYZE sí corrió (hay fecha en last_analyze) así que el motor tiene estadísticas frescas para decidir cómo resolver las consultas, aunque nunca haya recuperado espacio de filas borradas.

f) Espacio

```
SELECT pg_size_pretty(pg_database_size('sgbd_u5')) AS base,
       pg_size_pretty(pg_total_relation_size('sgbd_u5.movimiento')) AS tabla_con_indices,
       pg_size_pretty(pg_relation_size('sgbd_u5.movimiento')) AS solo_la_tabla;
```

Query

Query History

1

2

3

4

SELECT

pg_size_pretty(pg_database_size('sgbd_u5'))

AS base,

pg_size_pretty(pg_total_relation_size('sgbd_u5.movimiento'))

AS tabla_con_indices,

pg_size_pretty(pg_relation_size('sgbd_u5.movimiento'))

AS solo_la_tabla;

Data Output

Messages

Notifications

≡

+

📄

▼

📁

▼

🗑️

▼

📧

▼

📥

▼

📶

▼

SQL

Showing rows: 1 to 1

✎

🔍

	base	tabla_con_indices	solo_la_tabla
	text	text	text
1	32 MB	24 MB	14 MB

base: 32 MB · tabla_con_indices: 24 MB · solo_la_tabla: 14 MB

La tabla sola pesa 14 MB, pero con sus índices sube a 24 MB: los índices agregan **10 MB**, alrededor de un 70 % extra sobre el tamaño de la tabla. Confirma algo que se suele repetir pero no siempre se ve con un número al lado: los índices no son gratis, tienen un costo real de espacio a cambio de consultas más rápidas.

g) Mantenimiento

```
SELECT relname, n_live_tup, last_vacuum, last_autovacuum, last_analyze, last_autoanalyze
FROM pg_stat_user_tables
WHERE schemaname = 'sgbd_u5';
```

The screenshot shows a PostgreSQL query execution interface. The query executed is: `SELECT relname, n_live_tup, last_vacuum, last_autovacuum, last_analyze, last_autoanalyze FROM pg_stat_user_tables WHERE schemaname = 'sgbd_u5';`. The results are displayed in a table with 6 columns: relname, n_live_tup, last_vacuum, last_autovacuum, last_analyze, and last_autoanalyze. There are two rows of data.

	relname	n_live_tup	last_vacuum	last_autovacuum	last_analyze	last_autoanalyze
1	movimien...	200000	[null]	2026-09-15 21:56:45.806726+...	2026-09-15 21:56:10.292558+...	[null]
2	cuenta	4	[null]	[null]	2026-09-15 21:56:10.29574+00	[null]

movimiento

En ninguna de las dos tablas corrió nunca un **VACUUM**, ni manual ni automático (`last_vacuum` y `last_autovacuum` vacíos). **ANALYZE** sí corrió — hay fecha en `last_analyze` — así que el motor tiene estadísticas frescas para decidir cómo resolver las consultas, aunque nunca haya recuperado espacio de filas borradas.

h) Cierre

El servidor está sano en lo que pude verificar, con dos cosas para seguir de cerca. Primero, `last_vacuum` aparece vacío en movimiento y en cuenta: nunca se ejecutó un **VACUUM** manual, solo corrió **ANALYZE**. No es grave con la carga actual, pero conviene vigilarlo si la tabla sigue creciendo. Segundo, en movimiento los índices agregan 10 MB sobre los 14 MB de la tabla —un 70 % más—, un costo de espacio razonable a cambio de que `ix_mov_cuenta_fecha` acelere la consulta por cuenta y fecha. Falta incorporar los indicadores de rendimiento (cache hit, deadlocks, temp_files) del punto e) antes de dar el cierre por completo, ya que esa captura quedó pendiente de rehacer.

Conclusión

Conclusión grupal

Si mañana el servidor presenta problemas, revisariamos primero estos cinco elementos desde la herramienta:

1. **Sesiones activas (pg_stat_activity):** Para ver si hay conexiones colgadas, demasiados usuarios acumulados o consultas bloqueando a los demás.
2. **Dashboard de rendimiento (Cache Hit % y transacciones):** Nos muestra rápido si el motor está leyendo de memoria o si sufre yendo al disco, además de ver si subieron los *rollbacks*.
3. **Bloqueos y esperas (pg_locks / Wait Events):** Es clave para detectar si hay transacciones trabadas compitiendo por un mismo dato o un posible *deadlock*.
4. **Plan de ejecución (EXPLAIN ANALYZE):** Permite revisar por qué una consulta específica se volvió lenta o dejó de usar los índices.
5. **Espacio en disco y tablas:** Para chequear que el crecimiento de los datos o de los archivos temporales (*temp_files*) no esté llenando el almacenamiento de la VM.